Pflichtenheft

Das Verrückte Labyrinth

Advanced Integrative Project

Wintersemester 2026/2027

1. Einleitung **3**
1.1. Zweck 3
1.2. Geltungsbereich 3
1.3. Definitionen und Abkürzungen 4
2. Referenzen **6**
3. Produktüberblick **6**
3.1. Produktperspektive 6
3.2. Produktfunktionen (Überblick) 6
3.3. Benutzermerkmale 7
3.4. Einschränkungen 7
3.5. Abhängigkeiten 7
4. Spezifische Anforderungen **8**
4.1. Schnittstellen 8
4.2. Funktionale Anforderungen 10
4.3. Usability-Anforderungen 13
4.4. Leistungsanforderungen 14
4.5. Nichtfunktionale Anforderungen 14
4.5.1. Performance Anforderungen 14
4.5.2. Kompatibilitätsanforderungen 15
4.5.3. Zuverlässigkeitsanforderungen 15
4.5.4. Sicherheitsanforderung 15
4.5.5. Wartbarkeitsanforderungen 16
4.6. Entwurfsrandbedingungen 16
4.7. Systemattribute 16
5. Technische Realisierung **18**
5.1. Sicherheitskonzept 20
6. Verifikation und Abnahme **21**
6.1. Testarten 21
6.2. Abnahmekriterien 21
7. Projektorganisation **23**
7.1. Rollen 23
7.2. Meilensteine 24
7.3. Risiken 25
8. Anhang **28**
8.1. Änderungshistorie 28
8.2. Freigabe 28
8.3. Diagramme 28

1. Einleitung

Dieses Pflichtenheft beschreibt die technische und organisatorische Umsetzung der App “Das verrückte Labyrinth”. Als Ausgangsbasis dient das Lastenheft des Auftraggebers sowie das gleichnamige Gesellschaftsspiel des Spielwarenherstellers Ravensburger.

Ziel dieses Projekts ist die Entwicklung einer digitalen Version des originalen Spiels. Der grundlegende Spielablauf, das Spielfeld sowie die Spielregeln müssen gleich wie beim Original umgesetzt werden. Zusätzlich muss die digitale Version um weitere Funktionen ergänzt werden, welche den Spielspaß bei den Nutzern der App langfristig erhöhen sollen.

Eine zentrale Rahmenbedingung dieser App ist, dass Anwendungen anderer Anbieter des digitalen Spiels mit dieser App kompatibel sein müssen. Unabhängig davon, ob der entwickelte Server/Client des Auftragnehmers oder ein Server/Client eines anderen Anbieters verwendet wird, müssen alle Funktionen in vollem Umfang zur Verfügung stehen. Die Kommunikation zwischen den Anwendungen muss aufgrund gemeinsamer Schnittstellen ermöglicht werden. Dadurch soll sichergestellt werden, dass Spieler unabhängig von der zugrundeliegenden Anwendung das gleiche Spiel spielen können.

1.1. Zweck

Dieses Dokument dient als Vertragsgrundlage zwischen Auftraggeber und Auftragnehmer. Zudem dient es dem Auftragnehmer als Arbeitsgrundlage für die Entwicklung, den Test und die Abnahme der App. Da es sich um ein Studienprojekt für den Masterstudiengang “Digital Business and Software Engineering” handelt, dient dieses Pflichtenheft ebenso als Bewertungsgrundlage für das Modul “Advanced Integrative Project”.

1.2. Geltungsbereich

- **Produktname:** Das verrückte Labyrinth
- **Plattformen:** Web-App
- **Im Umfang enthalten:** Erstellen und Starten eines Spiels mit 2-4 Spielern, Durchführung von Spielzügen, Einsammeln von Boni und Achievements
- **Nicht im Umfang enthalten (Abgrenzung):** Offline-Modus, Spielerstatistiken
- **Nutzen für den Auftraggeber:** Die Studierenden lernen innerhalb eines Semesters ein technisch anspruchsvolles System zu designen, planen und implementieren. Dadurch lernen die Studierenden in einem selbstorganisierten Team zu arbeiten und sich zugleich regelmäßig mit dem Auftraggeber, ähnlich wie bei realen Projekten in der Wirtschaft, abzustimmen. Der Auftraggeber erhält ein fertiges Spiel, welches von zwei oder bis zu vier Spielern gleichzeitig gespielt werden kann.

1.3. Definitionen und Abkürzungen

Begriff	Bedeutung
Lastenheft	Gesamtheit der Forderungen des Auftraggebers Beantwortung folgender Fragen: Was soll umgesetzt werden? Wofür soll die Umsetzung erfolgen?
Pflichtenheft	Realisierungsvorgaben des Auftragnehmers auf Basis des Lastenhefts Beantwortung folgender Fragen: Wie soll die App umgesetzt werden? Womit soll die App umgesetzt werden?
API-Key	Geheimer Schlüssel eines Teams, mit dem sich dessen Spielserver am Verzeichnisserver anmeldet, Heartbeats sendet und abmeldet
Bonus (Boni)	Zeitgesteuert auf Kacheln erscheinender Gegenstand mit Sondereffekt (Beamen, Feste Gänge schieben, Tauschen, Zweimal schieben), pro Spiel nur einmal einsetzbar
Broadcast / Unicast	Nachricht des Spielservers an alle Spieler bzw. nur an einen einzelnen Spieler
Client	Anwendung des Spielers, die den Spielzustand darstellt und Befehle an den Spielserver sendet
Feature Freeze	Zeitpunkt (11.01.2027), ab dem keine neuen Funktionen mehr ergänzt und nur noch Fehler behoben werden
Go-Live	Offizielle Inbetriebnahme und Präsentation des Systems am 03.02.2027
Heartbeat	Regelmäßiges Lebenszeichen eines Spielservers an den Verzeichnisserver (alle 10 Sekunden)
Heimatecke	Startecke einer Spielfigur, zu der sie nach dem Finden aller Schätze zurückkehren muss
Kachel	Einzelnes Wegplättchen des Spielfelds, auch die Ersatzkachel, die in einen Gang eingeschoben wird
KI	Künstliche Intelligenz: vom Client gesteuerter virtueller Spieler, der den menschlichen Spieler vertreten kann
Kennungstoken	Geheimer Token, den der Spielserver bei der Anmeldung vergibt und der die Wiederanmeldung nach einem Verbindungsabbruch ermöglicht
Mock-Nachricht	Künstlich erzeugte Nachricht, mit der Client oder Server unabhängig voneinander getestet werden
Muss / Soll / Kann (M, S, K)	Priorität einer Anforderung (siehe Abschnitt 4)

Begriff	Bedeutung
Scope Creep	Schleichende Ausweitung des Projektumfangs durch zusätzliche Funktionen
Scrum-Lite	Reduzierte Form von Scrum mit Product Backlog, Sprints, Sprint Review und Qualitätssicherung am Ende jedes Sprints
Schnittstellen-komitee	Gremium aller teilnehmenden Gruppen, das die Schnittstellen zwischen Spielservern, Clients und Verzeichnisserver festlegt
Spielserver	Anwendung ohne Benutzeroberfläche, die ein Spiel verwaltet und alle Regeln prüft
Sprint / Sprint Review	Zeitlich begrenzte Arbeitsphase bzw. Überprüfung der Ergebnisse am Ende des Sprints
Verzeichnisserver	Zentraler Server, bei dem sich Spielserver anmelden und der Clients die Liste aller aktiven Spielserver liefert
FA	Funktionale Anforderung
UX	Usability-Anforderung (User Experience)
LA	Leistungsanforderung
PA	Performance-Anforderung
KA	Kompatibilitätsanforderung
IA	Interaktions- bzw. Integrationsanforderung (je nach Titel in 4.5.3)
RA	Zuverlässigkeitsanforderung (Reliability)
SA	Systemattribut
SI	Sicherheitsanforderung
WA	Wartbarkeitsanforderung
ERB	Entwurfsrandbedingung
SCH	Schnittstelle (V = Verzeichnisserver, C = Client → Server, S = Server → Client)
T / D / A / I	Verifikationsmethode: Test / Demonstration / Analyse / Inspektion
ISO/IEC/IEEE 29148, ISO/IEC 25030	Normen für Anforderungsspezifikation bzw. nichtfunktionale Anforderungen

2. Referenzen

Dokument	Version / Datum	Bemerkung
Lastenheft “Das verrückte Labyrinth”	v1.0, 02.09.2026	Grundlage dieses Pflichtenhefts

3. Produktüberblick

3.1. Produktperspektive

Bei der zu entwickelnden App handelt es sich um ein Spiel, welches derzeit nur auf einem physischen Spielbrett gespielt werden kann. Das analoge Spiel soll durch die App virtualisiert werden. Ziel ist es, eine Server- und eine Clientanwendung zu entwickeln. Ein von den Studierenden gewähltes Schnittstellenkomitee stellt zudem die Schnittstellen für alle Projektgruppen bereit, sodass alle Anwendungen miteinander kompatibel sind. Neben einer Server- und einer Clientanwendung gibt es zudem auch noch einen Verzeichnisserver, welcher vom Schnittstellenkomitee bereitgestellt wird und alle vorhandenen Serveranwendungen auflistet.

Das Problem der analogen Spielvariante ist, dass für den Aufbau des Spiels ausreichend Platz zur Verfügung stehen muss, um die Spielkarten entsprechend anordnen und verschieben zu können. Möchten Spieler das Spiel unterwegs, beispielsweise während einer Zug- oder Busfahrt spielen, ist oftmals keine geeignete Ablagefläche für das Spielfeld sowie die Spielkarten vorhanden. Ein weiteres Problem ist zudem, dass mindestens zwei bis vier Personen für das Spiel benötigt werden.

Das virtuelle Spiel soll es ermöglichen, dass ein Spieler das Spiel gegen einen oder mehrere KI-Spieler spielen kann. Darüber hinaus wird lediglich ein elektronisches Gerät mit einer Internetverbindung benötigt, um das Spiel zu spielen. Da es sich um eine Webanwendung handelt, ist es auch möglich, mit weit entfernten Freunden das Spiel zu spielen.

Möchte man als Spieler einen anderen Server- oder Client nutzen, kann beliebig auf andere Anwendungen gewechselt werden. Aufgrund der gemeinsamen Schnittstellen stehen alle Funktionen auf allen Anwendungen zur Verfügung.

3.2. Produktfunktionen (Überblick)

- Spiel erstellen
- Spieleranmeldung
- Spiel starten
- Spielzug ausführen
- Schatz aufsammeln
- Anzeigen des Punktestandes

3.3. Benutzermerkmale

Nutzergruppe	Beschreibung	Technische Kenntnisse	Häufigkeit der Nutzung
Spieler	Person, die das Spiel spielt	niedrig	Abhängig von der Spielmotivation
Administrator	Person, die sich als erstes angemeldet hat	mittel	Abhängig, von der Spielmotivation

3.4. Einschränkungen

- Abgabe des Pflichtenhefts am 25.10.2026
- Abgabe des Testplans und der Unit Tests am 22.11.2026
- Finale Abgabe am 24.01.2027
- Go-Live inklusive einer Projektpräsentation am 03.02.2027
- Vorgaben des Schnittstellenkomitees müssen eingehalten werden

3.5. Abhängigkeiten

- Abhängigkeit: Die Schnittstellen werden vom Schnittstellenkomitee bereitgestellt.
- Abhängigkeit: Der Verzeichnisserver wird vom Schnittstellenkomitee bereitgestellt.

4. Spezifische Anforderungen

Jede Anforderung ist nach ISO/IEC/IEEE 29148 eindeutig, überprüfbar und singulär (eine Aussage pro Zeile). Priorität: **M** = Muss, **S** = Soll, **K** = Kann. Verifikation: **T** = Test, **D** = Demonstration, **A** = Analyse, **I** = Inspektion.

4.1. Schnittstellen

ID	Schnittstelle	Beschreibung	Priorität	Verifikation
Verzeichnisserver *Sequenzdiagramm siehe Kapitel 8.3, S. 33*				
SCH-V-01	POST /api/servers	Spielserver registrieren	M	T
SCH-V-02	GET /api/servers	Aktive Spielserver auflisten	M	T
SCH-V-03	DELETE /api/servers/{id}	Spielserver abmelden	M	T
SCH-V-04	PUT /api/servers/{id}/heartbeat	Lebenszeichen und Spielstand aktualisieren	M	T
Client zu Spielserver (ws://{host}:{port}/game/) *Sequenzdiagramm Verbindungsaufbau siehe Kapitel 8.3, S. 35* *Sequenzdiagramm Spielzug siehe Kapitel 8.3, S. 34*				
SCH-C-01	CONNECT	Vom Client innerhalb von 10 Sekunden nach WebSocket-Verbindung zu senden.	M	T
SCH-C-02	DISCONNECT	Lobby oder Partie wird bewusst verlassen.	M	T
SCH-C-03	START_GAME	Vom Admin-Client sendbar um das Spiel aus der Lobby heraus zu starten.	M	T
SCH-C-04	ROTATE_SPARE_TILE	Wird vom aktuellen Spieler verwendet, um die überschüssige Kachel um 90° im Uhrzeigersinn zu drehen.	M	T
SCH-C-05	PUSH_TILE	Wird vom aktuellen Spieler verwendet, um die überschüssige Kachel in einen geeigneten Gang einzufügen.	M	T
SCH-C-06	MOVE_FIGURE	Wird vom aktuellen Spieler verwendet, um seine Spielfigur an eine gewünschte Position zu schieben.	M	T
SCH-C-07	PASS_MOVE	Wird vom aktuellen Spieler verwendet, um seine Spielfigur in einem Spielzug nicht weiter zu bewegen.	M	T

SCH-C-08	USE_BONUS	Wird vom aktuellen Spieler verwendet, um einen Bonus einzulösen.	M	T
SCH-C-09	SET_AI_MODE	Wird vom Client des aktiven Spielers an den Server gesendet, sobald der Spieler den AI-Modus aktiviert.	M	T
SCH-C-10	REMATCH_REQUEST	Nach Beendigung des Spiels haben die Clients die Möglichkeit, mit dieser Nachricht an den Server in der Spiellobby zu verbleiben. Senden Clients diese Nachricht nicht, werden sie aus der Lobby entfernt.	M	T
Spielserver zu Client *Sequenzdiagramm Verbindungsaufbau siehe Kapitel 8.3, S. 35* *Sequenzdiagramm Spielzug siehe Kapitel 8.3, S. 34*				
SCH-S-01	SERVER_INFO	Wird vom Spielserver sofort nach erfolgreichem WebSocket-Handshake an den Client versendet.	M	T
SCH-S-02	CONNECT_ACK	Spielserver bestätigt die Verbindung mit dem Client und übergibt dem Client seinen Identifikator für die Session.	M	T
SCH-S-03	LOBBY_STATE	Spielserver sendet Informationen über die Lobby an alle anwesenden Clients.	M	T
SCH-S-04	GAME_STARTED	Spielserver sendet eine kurze Nachricht an alle Spieler über das Layout des Spielbretts und die überschüssige Kachel. Ein vollständiges Update über den Game-State folgt erst im Anschluss.	M	T
SCH-S-05	NEXT_TREASURE	Der Server übergibt dem Client seinen als nächstes einzusammelnden Schatz.	M	T
SCH-S-06	GAME_STATE_UPDATE	Der Spielserver übermittelt allen Clients den aktuellen Spielstatus.	M	T
SCH-S-07	BONUS_SPAWNED	Spielserver informiert alle Clients über einen neu erschienenen Bonus am Spielbrett.	M	T
SCH-S-08	BONUS_COLLECTED	Spielserver informiert alle Clients darüber, dass ein Spieler einen Bonus eingesammelt hat.	M	T
SCH-S-09	BONUS_USED	Spielserver informiert alle Clients darüber, dass ein Spieler einen Bonus verwendet hat.	M	T

SCH-S-10	TREASURE_FOUND	Spielserver informiert alle Clients darüber, dass ein Spieler einen Schatz eingesammelt hat.	M	T
SCH-S-11	PLAYER_UPDATED	Spielserver informiert alle Clients darüber, dass sich der Verbindungs- oder AI-Status eines Spielers geändert hat.	M	T
SCH-S-12	ACHIEVEMENT_UNLOCKED	Spielserver informiert alle Clients darüber, dass ein Spieler ein Achievement freigeschaltet hat.	M	T
SCH-S-13	TIMEOUT_WARNING	Spielserver informiert alle Clients darüber, dass ein Spieler den Timeout erreicht.	M	T
SCH-S-14	PLAYER_DISCONNECTED	Spielserver informiert alle Clients darüber, dass ein Spieler aus dem Spiel entfernt wurde oder das Spiel verlassen hat.	M	T
SCH-S-15	GAME_OVER	Der Spielserver informiert alle Clients darüber, dass das Spiel beendet wurde und sendet eine Aufstellung der Punkte mit.	M	T
SCH-S-16	REMATCH_STATUS	Der Spielserver informiert nach Beendigung des Spiels alle Clients über den Status einer Revanche.	M	T
SCH-S-17	ERROR	Der Spielserver sendet Fehlernachrichten immer nur an den Verursacher, z. B. falls ein Spielzug nicht möglich ist, so wie es der Client möchte.	M	T

Die Schnittstellen zwischen Spielservern und Clients sowie die Schnittstellen des Verzeichnisservers werden zentral zwischen allen teilnehmenden Gruppen im Schnittstellenkomitee definiert und aktualisiert. Die Dokumentation des Schnittstellenkomitees ist unter folgender URL verfügbar: https://synpex.github.io/lab-protocol/. Als Ist-Zustand für die Schnittstellendokumentation ist immer nur die Dokumentation des Schnittstellenkomitees zu Rate zu ziehen, da sie, anders als dieses Pflichtenheft, gewartet und regelmäßig aktualisiert wird. Der Verzeichnisserver wird ebenfalls zentral von Mitgliedern des Schnittstellenkomitees implementiert und gewartet und so aktuell mit der Schnittstellendokumentation gehalten.

4.2. Funktionale Anforderungen

ID	Anforderung	Priorität	Verifikation
FA-01	Ein Spieler muss sich bei einem Spiel anmelden können, sobald der Spielserver gestartet wurde.	M	D

ID	Anforderung	Priorität	Verifikation
FA-02	Der Administrator muss das Spielfeld vor dem Spielstart konfigurieren können. FA-02.1 Spielfeldgröße m x n Felder FA-02.2 Spieleranzahl z FA-02.3 Schatzkartenanzahl x FA-02.4 Spielobjekte auf jedem Spielfeld platzierbar m, n, z und x sind konfigurierbar	M	D
FA-03	Der Administrator (=Spielleiter) muss ein neues Spiel starten können, sobald das Spielfeld konfiguriert worden ist.	M	D
FA-04	Ein Spieler muss eine Liste der verfügbaren Server sehen können, sobald ein Client gestartet wurde.	M	D
FA-05	Ein Spieler muss einen Benutzernamen angeben, sobald ein Server ausgewählt wurde.	M	D
FA-06	Der Client muss sich mit dem Server verbinden, sobald ein Benutzername von einem Spieler eingegeben wurde.	M	D
FA-07	Der Server oder der Administrator muss einen Spielstart auslösen können, wenn mindestens zwei oder höchstens vier Spieler angemeldet worden sind.	M	D
FA-08	Der Server muss das Spielfeld aufbauen und die Spielobjekte sowie Spielfiguren verteilen können, wenn ein Spiel gestartet worden ist.	M	D
FA-09	Der Server muss die Clients laufend informieren, wenn Änderungen am Spielfeld vorgenommen werden.	M	D
FA-10	Ein Spieler muss seine geheimen Missionskarten (Schatzkarten) erhalten, wenn ein Spiel gestartet wurde.	M	D
FA-11	Ein Spieler muss einen Spielzug ausführen können, wenn der jeweilige Spieler am Zug ist. Ein Spielzug besteht aus zwei Schritten: FA-11.1 Gang verschieben FA-11.2 Spielfigur verschieben	M	D
FA-12	Der Administrator muss eine Spielzeit konfigurieren können, sobald ein neues Spiel angelegt worden ist.	M	D
FA-13	Ein Spieler muss Schätze einsammeln können, wenn dieser auf einem Feld mit einem Schatz steht.	M	D
FA-14	Der Server muss ein Spiel beenden können, wenn die konfigurierte Spielzeit erreicht worden ist oder ein Spieler alle Schätze eingesammelt hat und zur Heimatecke zurückgekehrt ist.	M	T
FA-15	Der Server muss eine Rangliste und weitere Kennzahlen anzeigen können, wenn ein Spiel beendet wurde. FA-15.1 Spielerplatzierung FA-15.2 Geschobene Gänge FA-15.3 Gesammelte Schätze FA-15.4 Blockierte Wege FA-15.5 Zurückgelegte Schritte (Gesamt und pro Spieler)	M	D

ID	Anforderung	Priorität	Verifikation
FA-16	Der Server oder der Administrator muss ein neues Spiel starten können, wenn eine laufende Partie beendet wurde. Alle bereits angemeldeten Clients werden dem neuen Spiel automatisch hinzugefügt und können sich 15 s lang für das Bleiben, also die Revanche, entscheiden. Weitere Clients können dem Spiel beitreten, sobald Plätze frei sind.	M	D
FA-17	Der Server muss einen Client vom Spiel entfernen, wenn der Client länger als 30 Sekunden nicht auf eine Anfrage reagiert. Vor der Entfernung wird ein Countdown von drei Sekunden angezeigt.	M	T
FA-18	Der Server protokolliert den Spielverlauf in der Konsole, sobald ein Spiel gestartet wurde.	M	I
FA-19	Der Spieler muss sich durch eine KI ersetzen lassen können, sobald ein Spiel gestartet wurde.	M	D
FA-19.1	Die KI muss in mindestens 80 % ihrer Spielzüge die Spielfigur näher an das Zielfeld des aktuellen Schatzes bewegen.	M	T
FA-20	Boni: Der Spielserver muss in bestimmten zeitlichen Abständen Boni auf geeigneten Kacheln erscheinen lassen, die durch die Spieler eingesammelt werden können. Die Boni sind der Schnittstellendokumentation zu entnehmen.	M	T
FA-21	Achievements: Der Spielserver muss Spielern Achievements für besondere Leistungen verleihen, z. B. für mehrere Siege in Folge. Welche Kriterien Achievements erfüllen müssen, ist der Schnittstellendokumentation zu entnehmen.	M	T
FA-22	Der Administrator muss ein Spielzugzeitlimit konfigurieren können, sobald ein neues Spiel angelegt worden ist.	M	D
FA-23	Der Administrator muss das Intervall für das Erscheinen von Boni konfigurieren können, sobald ein neues Spiel angelegt worden ist.	M	D
FA-24	Der Server muss die Standardwerte für Spielfeldgröße, Schatzkartenanzahl, Spielzeit, Spielzugzeitlimit und Boni-Intervall voreinstellen, wenn der Administrator keine Werte angibt.	M	T
FA-25	Der Server muss einen Spieler warnen (TIMEOUT_WARNING), wenn dessen Spielzugzeitlimit fast erreicht ist.	M	T
FA-26	Der Server muss den aktiven Spielzug beenden und zum nächsten Spieler wechseln, wenn das Spielzugzeitlimit abgelaufen ist.	M	T

ID	Anforderung	Priorität	Verifikation
FA-27	Ein Spieler muss sich innerhalb von 30 Sekunden nach einem Verbindungsabbruch mit seinem Kennungstoken wieder verbinden können.	M	T
FA-28	Der Server muss einem wieder verbundenen Spieler den vollständigen Spielzustand und seine aktuelle Schatzkarte senden, sobald die Wiederanmeldung erfolgreich war.	M	T
FA-29	Der Spielserver muss sich am Verzeichnisserver mit dem API-Key seines Teams registrieren, sobald er gestartet wurde.	M	T
FA-30	Der Spielserver muss alle 10 Sekunden einen Heartbeat an den Verzeichnisserver senden, solange er läuft.	M	T
FA-31	Der Spielserver muss sich neu registrieren, wenn der Verzeichnisserver auf den Heartbeat mit 404 antwortet.	S	T
FA-32	Der Spielserver muss sich am Verzeichnisserver abmelden, bevor er heruntergefahren wird.	M	T
FA-33	Der Server muss einen Spieler zum Sieger erklären, wenn er alle seine Schätze gefunden hat und zu seiner Heimatecke zurückgekehrt ist.	M	T
FA-34	Ein Spieler muss einen eingesammelten Bonus einsetzen können, wenn er am Spielzug ist und den Bonus noch nicht verwendet hat.	M	T
FA-35	Der Server muss einen Bonus pro Spieler und Spiel auf einmalige Verwendung begrenzen.	M	T
FA-36	Der Server muss den am längsten anwesenden Spieler zum neuen Administrator machen, wenn der bisherige Administrator ausscheidet.	M	T
FA-37	Der Server muss weitere Beitrittsanfragen mit einer Fehlermeldung ablehnen, wenn bereits vier Spieler angemeldet sind.	M	T

4.3. Usability-Anforderungen

ID	Anforderung	Priorität	Verifikation
UX-01	Das Spielfeld muss übersichtlich dargestellt sein.	M	D

ID	Anforderung	Priorität	Verifikation
UX-02	Die Spielerfarben (Rot, Blau, Grün, Gelb) müssen zusätzlich durch Symbole unterscheidbar sein, damit sie auch für Farbenblinde erkennbar sind.	M	D
UX-03	Der aktive Spieler und die nächste mögliche Aktion (Kachel einschieben oder Figur bewegen) müssen jederzeit klar erkennbar sein.	M	D
UX-04	Der Client muss unzulässige Spielzüge mit einer verständlichen Fehlermeldung quittieren, die aus der ERROR-Nachricht des Servers abgeleitet wird.	M	T
UX-05	Ein Spielzug soll mit wenigen Klicks durchführbar sein.	S	D
UX-06	Die Bedienung soll für Personen, die das Brettspiel kennen, ohne Anleitung verständlich sein.	S	D

4.4. Leistungsanforderungen

ID	Anforderung	Priorität	Verifikation
LA-01	Der Spielserver muss einen Spielzug in unter 1 Sekunde validieren.	M	T
LA-02	Der Spielserver muss Änderungen innerhalb von 2 Sekunden an alle Clients übertragen.	M	T
LA-03	Der Spielserver muss 4 gleichzeitige Spieler ohne spürbare Verzögerung verwalten, auch bei der größten Spielfeldkonfiguration.	M	T
LA-04	Die KI des Clients soll pro Spielzug höchstens 5 Sekunden benötigen.	S	T

4.5. Nichtfunktionale Anforderungen

4.5.1. Performance Anforderungen

ID	Anforderung	Priorität	Verifikation
PA-01	Spielzugvalidierung und -verarbeitung dauern höchstens 500 ms bei einem Maximum von 4 Spielern	S	T

ID	Anforderung	Priorität	Verifikation
PA-02	Zustandsänderungen/Spielzüge werden an alle Clients in 1000ms übermittelt	S	T
PA-03	Spielzüge durch die KI dauern höchstens 1000 ms/s pro Spielzug.	S	T
PA-04	Die Validierung, ob der Spielzug gültig war, dauert maximal 250ms	S	T
PA-05	Es dürfen nicht mehr als 100 MB Speicher reserviert werden auf dem Server je Partie.	S	T
PA-06	Es muss bis zu 1 Partie gleichzeitig laufen können und die anderen Anforderungen müssen erfüllt sein	S	T

4.5.2. Kompatibilitätsanforderungen

ID	Anforderung	Priorität	Verifikation
KA-01	Das Kommunikationsprotokoll muss entsprechend der Schnittstellendefinition umgesetzt sein und darf keine eigens definierten Funktionen beinhalten.	M	T
KA-02	Alle Nachrichten müssen dem Kommunikationsprotokoll entsprechen. Abweichungen werden ignoriert und verworfen.	M	T
KA-03	Die Schnittstelle vom Server und Kommunikationsprotokoll ist vollständig dokumentiert. Fehlerszenarien sind bekannt und werden abgehandelt.	S	T

4.5.3. Zuverlässigkeitsanforderungen

ID	Anforderung	Priorität	Verifikation
RA-01	Ein Client wird nach einer Inaktivität von 30 s und Countdown von 3 s entfernt.	M	T
RA-02	Bei Ausfall eines Clients (Ausfall, Rauswurf oder Abbruch) läuft für die anderen Spieler die Partie fehlerfrei weiter Fehlertoleranz: In 100 % der definierten Ausfallszenarien (Client-Ausfall, Rauswurf, Abbruch) läuft die Partie der übrigen Spieler fehlerfrei weiter.	S	T
RA-03	Für 30 s ist eine Neuverbindung des Clients möglich.	S	T

ID	Anforderung	Priorität	Verifikation
RA-04	Fehlerhafte Nachrichten werden zu 100% abgelehnt ohne dass es zu Performanceeinbußen führt Robustheit: 100 % der ungültigen oder manipulierten Nachrichten des Fuzz- und Protokolltestsatzes werden abgelehnt, ohne dass Serverzustand oder -prozess leiden.	M	T

4.5.4. Sicherheitsanforderung

ID	Anforderung	Priorität	Verifikation
SI-01	Aktionen werden eindeutig einem Spieler zugeordnet.	M	T
SI-02	Fremde Schatzkarten sind nicht für andere Spieler sichtbar.	M	T
SI-03	Rechenschaftspflicht: 100 % der Aktionen, auch abgelehnte Spielzüge, haben einen Protokolleintrag (Zeit, Spieler, Aktion, Ergebnis).	M	T

4.5.5. Wartbarkeitsanforderungen

ID	Anforderung	Priorität	Verifikation
WA-01	Die einzelnen Schichten haben keine Abhängigkeiten untereinander, nur die definierten Schnittstellen.	M	T
WA-02	Die APIs sind zu 100% dokumentiert.	M	A
WA-03	Mithilfe von statischer Codeanalyse werden die Regeln von guter Codequalität eingehalten.	S	T
WA-04	Das Protokoll wird persistent abgelegt, und ein Replay könnte die Partie zu 100 % identisch reproduzieren.	K	T

4.6. Entwurfsrandbedingungen

ID	Bedingung	Anforderung	Verifikation
ERB-01	Programmiersprache	Das System muss in Java implementiert werden	I

4.7. Systemattribute

ID	Attribut	Anforderung	Verifikation
SA-01	Zuverlässigkeit	Fehlerhafte oder manipulierte Nachrichten dürfen den Spielserver nicht zum Absturz bringen. Sie werden abgelehnt und protokolliert.	T
SA-02	Wiederherstellung	Ein Client muss sich nach Verbindungsabbruch innerhalb von 30 Sekunden mit seinem Token wieder verbinden und den vollständigen Spielzustand erhalten können.	T
SA-03	Sicherheit	Alle Spielregeln müssen ausschließlich serverseitig geprüft werden. Dem Client wird nie vertraut.	T
SA-04	Sicherheit	Der Server darf jedem Client nur dessen eigene geheime Schatzkarte senden.	T
SA-05	Sicherheit	Der Server muss Benutzernamen auf Länge (3–20 Zeichen) und erlaubte Zeichen prüfen und doppelte Namen ablehnen.	T/D
SA-06	Sicherheit	Registrierung, Heartbeat und Abmeldung am Verzeichnisserver müssen mit dem API-Key des eigenen Teams erfolgen. Der Key darf nicht an Clients weitergegeben werden.	T
SA-07	Datenschutz	Das System darf nur Benutzername und Sitzungstoken verarbeiten. Es gibt keine Benutzerkonten und keine Passwörter.	I/D
SA-08	Kompatibilität	Server und Client müssen die Nachrichten (JSON, Endpunkt /game, alle Nachrichtentypen) exakt nach der Dokumentation des Schnittstellenkomitees verwenden, damit sie mit Systemen anderer Gruppen zusammenarbeiten.	T
SA-09	Nachvollziehbarkeit	Der Spielserver muss Eingaben, Prüfschritte und Ausgaben jeder Partie in einem Logfile protokollieren.	D
SA-10	Testbarkeit	Zentrale Serverfunktionen, insbesondere unzulässige Spielzüge und unzulässige Client-Zugriffe, müssen durch automatisierte Tests abgesichert sein.	I
SA-11	Wartbarkeit	Der Code soll in den vier Modulen Server, Client, Message und Domain strukturiert und dokumentiert sein, die Dokumentation soll Diagramme enthalten.	I

ID	Attribut	Anforderung	Verifikation
SA-12	Portabilität	Das System muss auf der festgelegten Java-Version (25) und auf gängigen Betriebssystemen laufen. Der Standard-Client läuft im Desktop-Browser und ist nicht für Smartphones ausgelegt.	T/D

5. Technische Realisierung

Dieser Abschnitt hält fest, wie der Auftragnehmer die Anforderungen aus Abschnitt 4 umsetzt.

Bereich	**Entscheidung**	**Begründung**
Java-Version und Build-Tool	Es wird Java 25 mit dem Package Manager / Build-Tool Apache Maven verwendet	Java 25 ist die aktuellste Long-Term-Support-Version von Java, läuft somit stabil und wird aktiv gepflegt. Maven aufgrund vorhandenen Know-Hows SA-12 ERB-01
App-Architektur	Client-Server-Architektur in Bezug auf Verzeichnisserver, Spielserver und Client; MVC-Pattern auf Spielserver-Client-Ebene, wobei der Spielserver das Model verwaltet, der Client den View bereitstellt und sich Client und Spielserver die Controller-Rolle teilen (Client erzeugt Input, Server validiert und ändert Model) *Architektur-Diagramm siehe Kapitel 8.3, S. 29*	Erhöhte Test- und Wartbarkeit durch verteilte Verantwortlichkeiten; Möglichkeit verschiedene Komponenten gleichzeitig zu implementieren
Technologie Frontend	Vaadin Framework	Vorhandenes Know-how, gut integrierbar durch Spring-Boot, Portabilität ERB-01
Backend	Spring mit Spring-Boot mit WebSocket und REST API	Vorhandenes Know-how, notwendig aufgrund der Gruppen-übergreifend definierten Schnittstellen ERB-01
Datenbank	Es wird keine Datenbank benötigt	Es werden keine Daten dauerhaft gespeichert. Zu erstellende Logs werden als Log Files lokal gespeichert.
Authentifizierung	API-Key, Tokens	Der Spielserver weist sich gegenüber dem Verzeichnisserver mittels API-Key aus, der vom Verzeichnisserver generiert wird. Der Client weist sich gegenüber dem Spielserver mittels eines Tokens aus, der vom Spielserver generiert wird. SA-07

Hosting	Lokales Hosting vorgesehen; Hosting bei prinzipiell jedem Anbieter möglich	Der Spielserver soll von Spielern im lokalen Netzwerk auffindbar sein, kann aber auch über das Internet verfügbar gemacht werden. Da keine persönlichen Daten verarbeitet werden, spricht nichts gegen Hosting außerhalb der EU.
CI/CD	CI wird durch Unit- und Integrationstests umgesetzt, auf CD wird verzichtet	Es ist vorgesehen das System im lokalen Netzwerk direkt auf Endgeräten auszuführen
Monitoring	Befehle, die vom Spielserver ausgeführt bzw. empfangen werden, werden mittels Logging Library aufgezeichnet	Nachweis von SA-01 und SA-09. Das Protokoll ist jederzeit sichtbar.
Systemmodule	Client und Spielserver werden von jeder Gruppe implementiert. Diese beiden übergeordneten Teile des Systems werden in vier Java-Module aufgeteilt: Client-Modul, Spielserver-Modul, Domain-Modul und Messaging-Modul	Client und Spielserver sind teilweise gleich aufgebaut und folgen in ihren Grundzügen den gleichen Verhaltensmustern. Durch den modularen Aufbau kann Spiellogik und Messaging einmalig im Domain- und Messaging Modul implementiert und von dort aus wiederverwendet werden. Sämtliche Anforderungen an die Beschaffenheit von Nachrichten (SCH-V, SCH-C, SCH-S) werden durch das Messaging-Modul abgedeckt
Client-Modul	Zuständig für die Darstellung und Aktualisierung des UI und die Entgegennahme des Spieler- bzw. KI-Inputs *Klassendiagramm siehe Kapitel 8.3, S. 30*	Die Darstellung und die KI liegen im Client, die Regelprüfung beim Server (SA-03). Diese Trennung von Eingabe, Modell und Verbindung macht die Tests für den Client übersichtlicher und einfacher, auch mit Mock-Nachrichten.
Spielserver-Modul	Zuständig für die Verwaltung des Spiels und der Spieler-Sitzungen; bietet nur ein CLI und kein grafisches UI; Loggt Inputs, Schritte zur Bestimmung des Outputs und den Output selbst *Klassendiagramm siehe Kapitel 8.3, S. 32*	Befehle sind klar definiert und laufen sequentiell ab, deshalb bleibt der Server schlank. Ein CLI genügt für Konfiguration, Steuerung und die Anzeige des Protokolls.

Domain-Modul	Fasst alle Logik und Komponenten in sich zusammen, die sowohl der Client als auch der Server benötigen. Z. B. Logik zur Validierung von Spielzügen, die vom Client-KI Modus und vom Spielserver gebraucht wird. *Klassendiagramm siehe Kapitel 8.3, S. 31*	Durch Vereinheitlichung beinhalten Klassen an beiden Enden dieselben Informationen und verhalten sich gleich, was die Entwicklung beschleunigt.
Messaging-Modul	Bildet die Schnittstellen (SCH-V, SCH-C, SCH-S) mithilfe von Java-Klassen und Interfaces ab.	Ermöglicht automatisches Testen der Interfaces, u. A. durch Mocking.

5.1. Sicherheitskonzept

- Der Spielserver prüft alle Regeln selbst und vertraut den Clients nie. Unzulässige Befehle führen zu einer ERROR-Nachricht an den Verursacher, der Spielzustand bleibt unverändert.
- Geheime Schatzkarten werden ausschließlich einzeln an den berechtigten Spieler gesendet (NEXT_TREASURE), nie als Broadcast.
- **Rollen:** Spieler und Administrator. Der am längsten anwesende Spieler ist Administrator, nur er darf das Spiel starten. Scheidet er aus, rückt der nächste nach.
- Die Wiederanmeldung nach einem Verbindungsabbruch erfolgt über einen geheimen Kennungstoken. Ohne gültigen Token ist kein Beitritt zu einer laufenden Partie möglich.
- Der Zugriff auf den Verzeichnisserver ist pro Team über einen geheimen API-Key geschützt, der nur im Spielserver liegt. Im Produktivbetrieb wird HTTPS verwendet.
- Es gibt keine Benutzerkonten und keine Passwörter. Zertifikats-Pinning ist nicht vorgesehen.

6. Verifikation und Abnahme

6.1. Testarten

Teststufe	Inhalt	Verantwortlich	Zeitpunkt
Unit-Tests	Einzelne Funktionen und Klassen, insbesondere serverseitige Regelprüfung (unzulässige Spielzüge, ungültige Nachrichten)	Entwicklung	laufend, in jedem Sprint
Integrationstests	Zusammenspiel von Client, Spielserver und Verzeichnisserver, mit Mock-Nachrichten auch unabhängig voneinander	Entwicklung	pro Sprint
Kompatibilitätstests	Zusammenspiel mit Servern und Clients anderer Gruppen	Entwicklung, Schnittstellenvertreter (Anton)	ab Sprint 3, Generalprobe vor dem Go-Live
Systemtest	Alle Muss-Anforderungen aus Abschnitt 4 gegen die Testfälle	Qualitätsmanagement (Eva, Vertretung: Daniel)	Testphase 12.01.–23.01.2027
Abnahmetest	Prüfung durch den Auftraggeber	Auftraggeber	Abschlusspräsentation am 03.02.2027

Die Qualitätssicherung begleitet die gesamte Entwicklung. Neue Funktionen werden im jeweiligen Sprint durch Unit Tests abgesichert, im Sprint Review wird der aktuelle Teststand geprüft. Durch die Überführung der Nachrichten in Java-Klassen (siehe Kapitel 5) können Client und Server mit Mock-Nachrichten unabhängig voneinander getestet werden. Da das System keine Datenbank verwendet, entfallen Datenbanktests. Fehler werden in Jira erfasst, priorisiert und nachverfolgt. Die Koordination liegt bei Eva (Qualitätsmanagement, Vertretung: Daniel).

Der detaillierte Testplan samt Unit-Test-Gerüst wird bis 22.11.2026 abgegeben. Ab dem Feature Freeze am 11.01.2027 werden nur noch Fehler behoben und keine neuen Funktionen mehr ergänzt. Die abschließende Testphase mit Integrations- und Kompatibilitätstests, Bugfixing und Dokumentation findet vom 12.01. bis 23.01.2027 statt.

6.2. Abnahmekriterien

Das Projekt gilt als erfolgreich abgeschlossen, wenn die folgenden Kriterien bei der Abschlusspräsentation am 03.02.2027 nachgewiesen werden. Dabei wird das lauffähige Gesamtsystem gemeinsam mit den Systemen anderer Gruppen vorgeführt. Der Auftraggeber muss dafür Sorge tragen, dass ein anderes System zur Verfügung steht, um die Interoperabilität gewährleisten zu können.

Allgemein

- Alle Muss-Anforderungen (M) sind erfüllt und getestet.
- Es gibt keine offenen Fehler der Schwere „kritisch“ oder „hoch“.
- Die Leistungsziele aus 4.4 (LA-01 bis LA-03) sind nachgewiesen.

Gesamtsystem und Kompatibilität

- Verzeichnisserver-Anbindung, Spielserver und mindestens ein Java-Client sind lauffähig.
- Der Spielserver funktioniert mit Clients anderer Gruppen, und der Client funktioniert mit Spielservern anderer Gruppen.
- Der Client zeigt die verfügbaren Spielserver an, der Spieler wählt einen aus, gibt einen Benutzernamen an und meldet sich an.
- Es nehmen mindestens 2 und höchstens 4 Spieler teil, weitere Spieler werden abgelehnt.

Spielfeld und Konfiguration

- Die Spielfeldgröße (ungerade, mindestens 3), die Anzahl der Schatzkarten je Spieler, die Spieldauer, das Spielzugzeitlimit und das Boni-Intervall sind beim Spielstart konfigurierbar, sinnvolle Standardwerte sind voreingestellt.
- Jedes Feld trägt genau eine Kachel.

Spielregeln

- Der Server setzt alle Spielregeln des Lastenhefts und des Originalspiels durch und lehnt unzulässige Speilzüge ab.
- Ein Spieler gewinnt, wenn er alle seine Schätze gefunden hat und zur Heimatecke zurückgekehrt ist.
- Alle vier Boni (Beamen, Feste Gänge schieben, Tauschen, Zweimal schieben) funktionieren und sind pro Spiel nur einmal einsetzbar.

Spielende und Betrieb

- Die Partie endet bei Zeitablauf oder wenn ein Spieler alle Schätze gesammelt hat und zurück ist. Die Rangliste zeigt Punkte, Schritte, geschobene Kacheln, Schätze und blockierte Wege.
- Eine Revanche ist möglich, die bisherigen Teilnehmer haben dafür 15 Sekunden Vorrang.

- Der Spielverlauf wird je Partie in einem Logfile protokolliert.
- Ein Spieler, dessen Verbindung abbricht, kann sich innerhalb von 30 Sekunden wieder verbinden, danach wird er aus der Partie entfernt.
- Jeder Client bietet eine KI, die den Spieler vertritt und nicht rein zufällig agiert.

Qualität

- Zentrale Serverfunktionen sind durch Tests abgesichert, insbesondere unzulässige Spielzüge und unzulässige Client-Zugriffe.
- Entwurfsdokumente mit Diagrammen und eine Code-Dokumentation liegen vor.

7. Projektorganisation

7.1. Rollen

Rolle	Name	Aufgabe
Projektleitung Auftraggeber	Prof. Matthias Janetschek, PhD	Freigaben, Entscheidungen
Projektleitung Auftragnehmer	Daniel Weilguny, BSc	Planung, Steuerung
Entwicklung	Anton Wörndle, BSc Daniel Weilguny, BSc Daniela Schwarz, BA Eva-Marie Winkler, BSc Julian Högerl, BSc Sebastian Schiefermüller, BSc	Umsetzung, Entwicklung
Schriftführerin	Daniela Schwarz, BA	Protokollierung
Schnittstellenvertreter	Anton Wörndle, BSc	Vertretung des Teams im Schnittstellenkomitee
Qualitätsmanagerin	Eva-Marie Winkler, BSc	Sicherstellung der Qualität des Quellcodes und der Einhaltung der Anforderungen

Vorgehensmodell

Das Projekt wird nach Scrum-Lite durchgeführt, einer reduzierten Form von Scrum mit Product Backlog, Sprints, Sprint Review und Qualitätssicherung am Ende jedes Sprints. Gegenüber einem rein flussorientierten Kanban-Vorgehen bietet Scrum-Lite durch feste Sprints einen klaren zeitlichen Rhythmus und überprüfbare Zwischenziele, was bei festen Abgabeterminen von Vorteil ist. Die Reviews stellen sicher, dass Arbeitsergebnisse vor dem nächsten Sprint auf ihre Qualität geprüft werden, und dienen als Grundlage für die Abstimmung mit dem Betreuer. Auf weitere Scrum-Rituale wird bewusst verzichtet, um den Aufwand für das kleine Team gering zu halten. Die Sprints orientieren sich an den Vorlesungsterminen und an den Modulen aus Kapitel 5.

Besprechungsrhythmus

Das Team trifft sich anfangs wöchentlich im Anschluss an die Vorlesung. Der Termin der ersten Besprechung mit dem Betreuer wird von Daniel abgestimmt, die Rollenverteilung wird dem Betreuer vorab mitgeteilt. Der Rhythmus wird bei Bedarf angepasst. Anton vertritt das Team im Schnittstellenkomitee und trägt Änderungen an der Schnittstelle ins Team zurück.

Eingesetzte Tools

Zweck	Werkzeug
Aufgabenverwaltung (Backlog, Sprints, Fehler)	Jira
Versionsverwaltung, Maven-Projekt mit den Modulen Server, Client, Message, Domain	GitHub
Entwicklungsumgebung	Java, Maven, Spring Boot, Vaadin (siehe Kapitel 5)
Skizzen und Entwurfsdiagramme	draw.io, Mermaid.live
Schnittstellendokumentation	Dokumentation des Schnittstellenkomitees (siehe Kapitel 5)
Test der REST-Schnittstelle des Verzeichnisservers	Postman

7.2. Meilensteine

Meilenstein	Inhalt	Termin
M1	Abgabe Pflichtenheft	25.10.2026
M2	Projekt-Setup und Architektur: Repository mit Maven-Modulen, Entwicklungsumgebung bei allen eingerichtet, Jira-Board	10.11.2026
M3	Präsentation und Fragenrunde Pflichtenheft	11.–12.11.2026
M4	Abgabe Testplan und Unit-Test-Gerüst	22.11.2026
M5	Sprint 1 (Grundgerüst): Message-Modul (alle Nachrichtentypen), Domain-Modul, Anbindung des Verzeichnisservers	25.11.2026
M6	Präsentation und Fragenrunde Testplan	02.–03.12.2026

Meilenstein	Inhalt	Termin
M7	Sprint 2 (Server-Kernfunktionen): WebSocket-Endpunkt, Verbindungsaufbau, Lobby, Spielzugablauf, serverseitige Validierung, Client-Views für Serverliste, Lobby und Spielfeld	09.12.2026
M8	Lauffähiger Prototyp: vollständiger Spielzug von Client über Spielserver bis Spielzustandsupdate, Schätze und Spielende	15.12.2026
M9	Zwischenpräsentation	16.12.2026
M10	Sprint 3 (Erweiterungen): Boni, Reconnect, Speilzug-Timeout, Rangliste, Revanche, KI im Client, Logging pro Partie, erste Testläufe mit anderen Gruppen	04.01.2027
M11	Sprint 4 (Restfunktionen) und Feature Freeze: Achievements, UI-Feinschliff, Behebung von Befunden aus den Testläufen	11.01.2027
M12	Ende der Testphase (Integrations- und Kompatibilitätstests, Bugfixing, Dokumentation)	23.01.2027
M13	Bereitstellung der ersten Version (lauffähiges Gesamtsystem)	24.01.2027
M14	Go-Live und Abschlusspräsentation mit Kompatibilitätsdemo	03.02.2027

Fest vorgegeben sind die Abgabe- und Präsentationstermine, dazwischen plant das Team die Arbeitsphasen. Nach jedem Sprint wird der Zeitplan im Review überprüft und bei Bedarf angepasst, die Zwischenmeilensteine sind daher eine erste Planung. Muss-Funktionen werden vor Kann-Funktionen umgesetzt. Da Sprint 3 in die Weihnachtsferien fällt, wird dort mit geringerer Verfügbarkeit gerechnet und Kann-Funktionen dienen als Puffer. Die Abnahme erfolgt bei der Abschlusspräsentation am 03.02.2027, wo das System gemeinsam mit den Systemen anderer Gruppen vorgeführt und anhand der Muss-Kriterien aus Abschnitt 6.2 geprüft wird.

7.3. Risiken

Risiko	Wahrscheinlichkeit	Auswirkung	Gegenmaßnahme
Zeitmangel: Neben anderen Lehrveranstaltungen bleibt zu wenig Zeit, besonders in den Weihnachtsferien (Sprint 3).	hoch	hoch	Pufferzeiten im Zeitplan, Sprint Reviews, Priorisierung nach Muss/Kann

Risiko	Wahrscheinlichkeit	Auswirkung	Gegenmaßnahme
Fehlendes Know-how: Wissen zu Spring Boot, Vaadin, WebSocket und Spielregeln ist im Team ungleich verteilt.	mittel	mittel	Wissensaustausch, Pair Programming, früher Prototyp (M8)
Eingeschränkte Verfügbarkeit durch Krankheit oder Terminüberschneidungen.	mittel	mittel	Vertretungsregelung für Rollen, gute Dokumentation, kleine Aufgabenpakete
Scope Creep: Zusätzliche, nicht vereinbarte Funktionen (z. B. Erweiterungen bei Achievements, UI-Extras oder eigene Protokollerweiterungen) gefährden die Muss-Anforderungen und die Kompatibilität mit anderen Gruppen.	mittel	hoch	Umsetzung strikt nach Priorität (Muss vor Soll vor Kann), neue Funktionen nur nach Abstimmung im Team und Eintrag im Jira-Backlog, Achievements zunächst in minimaler Form gemäß Schnittstellendokumentation, Erweiterungen erst nach dem Feature Freeze-Plan (M11)
Langsame Einigung im Schnittstellenkomitee blockiert Server, Client und Tests.	mittel	hoch	Anton vertritt das Team aktiv, Annahmen dokumentieren, Schnittstelle in Message-Modul kapseln
Schnittstellenänderungen während der Entwicklung, da die Dokumentation laufend aktualisiert wird.	hoch	mittel	Message-Modul als zentrale Stelle für Anpassungen, Änderungen im Sprint Review prüfen
Inkompatibilität mit anderen Gruppen: Der Standard wird unterschiedlich interpretiert.	mittel	hoch	Frühe gemeinsame Testläufe ab Sprint 3, Generalprobe vor dem Go-Live, Protokollspezifikation schriftlich festhalten
Fehlerhafte Regelprüfung: Unzulässige Spielzüge werden nicht erkannt, Spielzustände laufen auseinander.	mittel	hoch	Umfangreiche Unit-Tests für die Regeln, ausschließlich serverseitige Validierung
Mängel in den Schnittstellen: Fehlende	mittel	mittel	Früher Unit-Test der Messaging-Komponenten,

Risiko	Wahrscheinlichkeit	Auswirkung	Gegenmaßnahme
oder nicht verfügbare Daten (z. B. im Request-Body) verhindern die Umsetzung.			Entwicklung der Hauptkomponenten parallel oder nachgelagert
Ausfall oder Neustart des Verzeichnisservers: Alle Registrierungen gehen verloren (nur im Arbeitsspeicher).	mittel	mittel	Spielserver registriert sich bei 404 auf den Heartbeat automatisch neu, Direktverbindung zum Spielserver als Rückfall beim Testen

8. Anhang

8.1. Änderungshistorie

Version	Datum	Autor	Änderung
0.1	11.09.2026	Eva Marie Winkler	Erstellung des Dokuments
0.2	07.10.2026	Team	Erstellung Skelett
0.3	18.10.2026	Team	Sammlung aller Inhalte
1.0	24.10.2026	Team	Vertiefung der Inhalte
1.1	24.10.2026	Team	Überarbeitungen
2.0	25.10.2026	Team	Finalisierung aller Inhalte, Konsistenzcheck

8.2. Freigabe

Rolle	Name	Datum	Unterschrift
Auftraggeber	Prof. Matthias Janetschek, PhD	03.02.2027	
Auftragnehmer	Daniel Weilguny, BSc	03.02.2027	

8.3. Diagramme

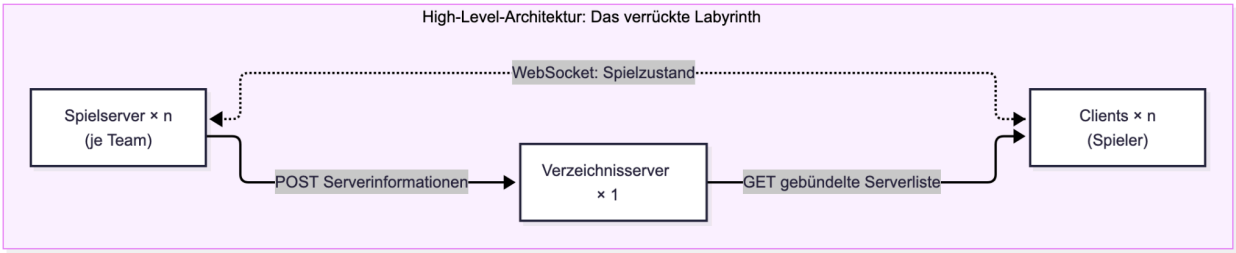
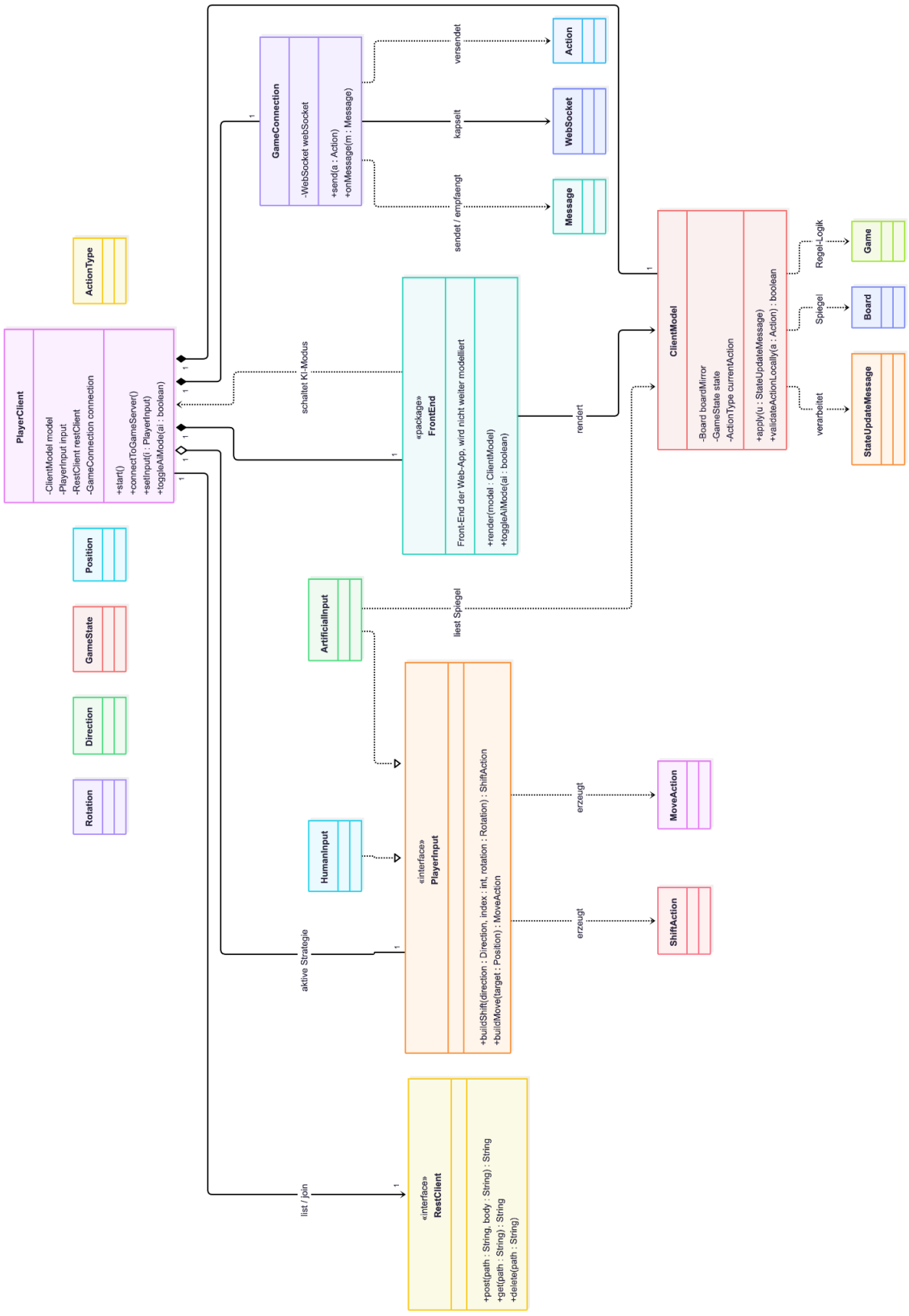
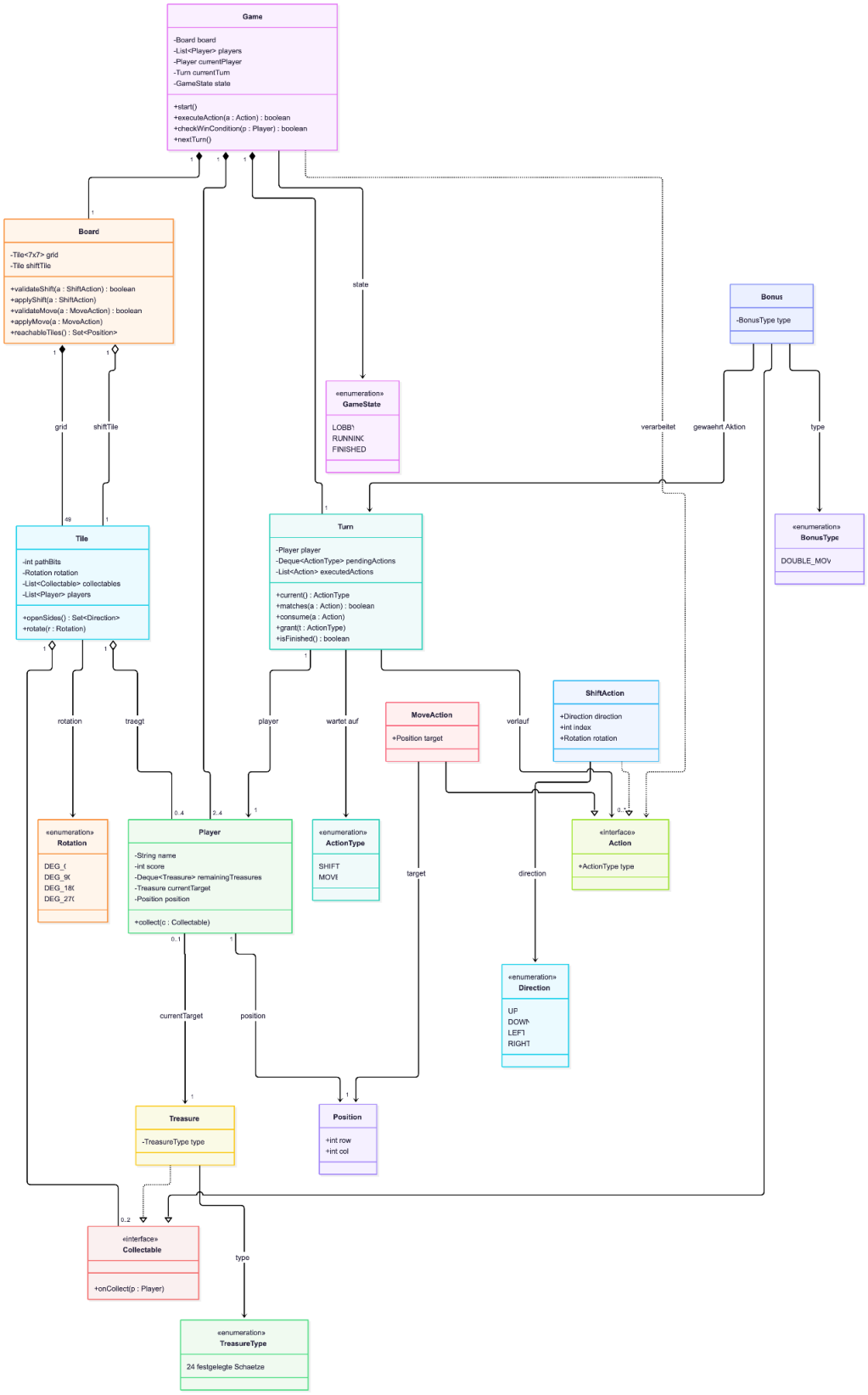


Diagramm Architektur

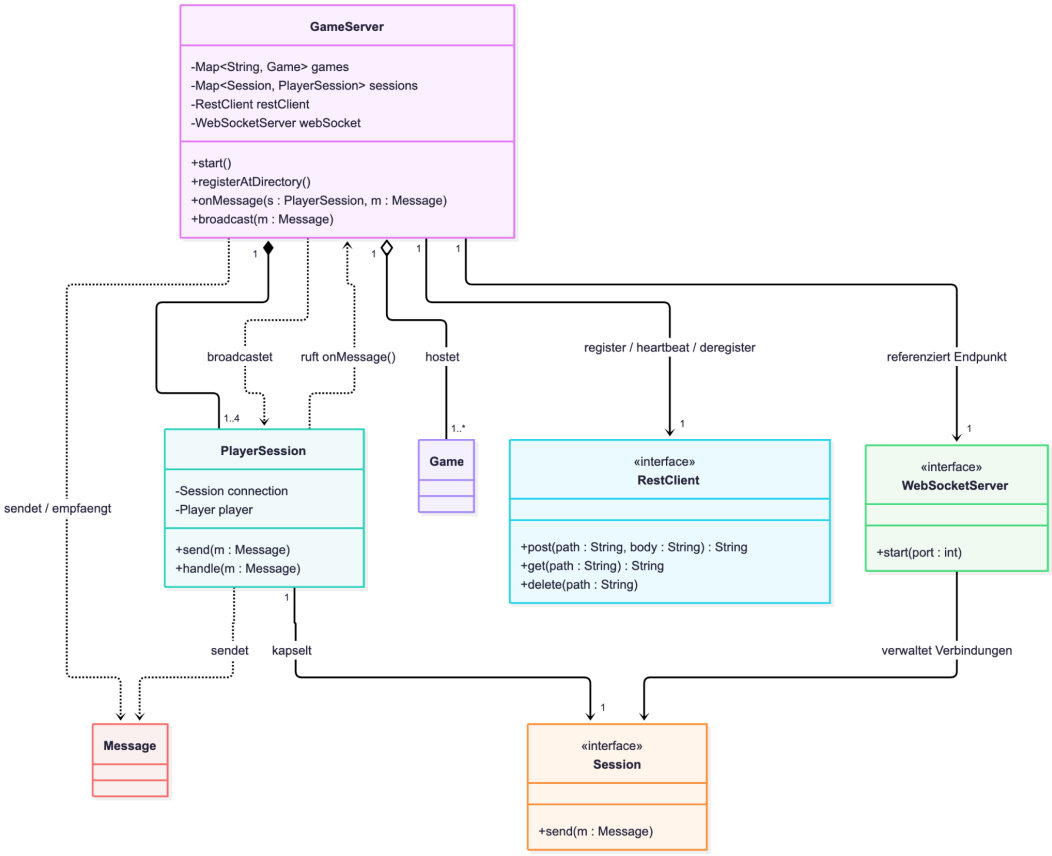
Klassendiagramm Client-Modul

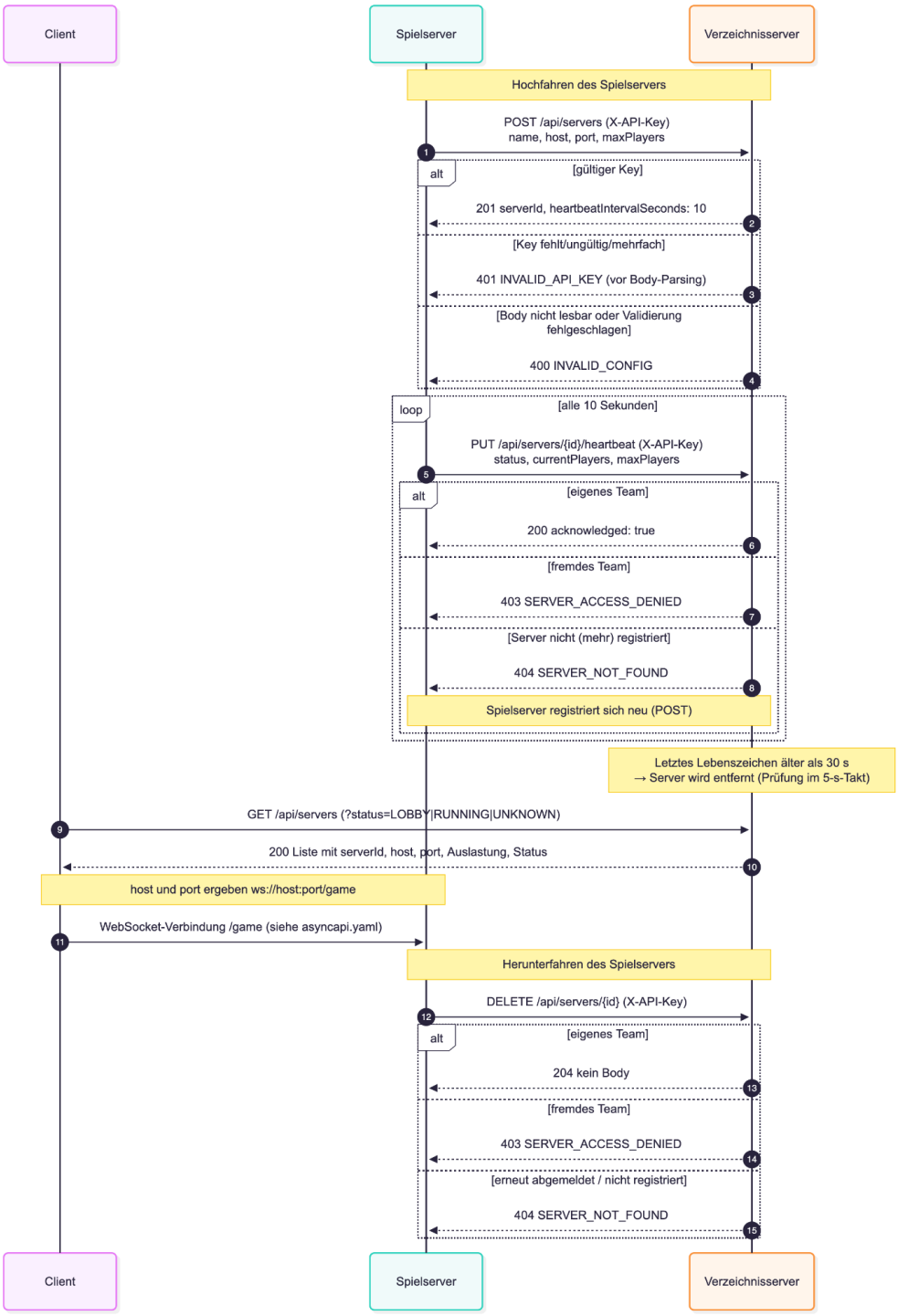


Klassendiagramm Domain-Modul

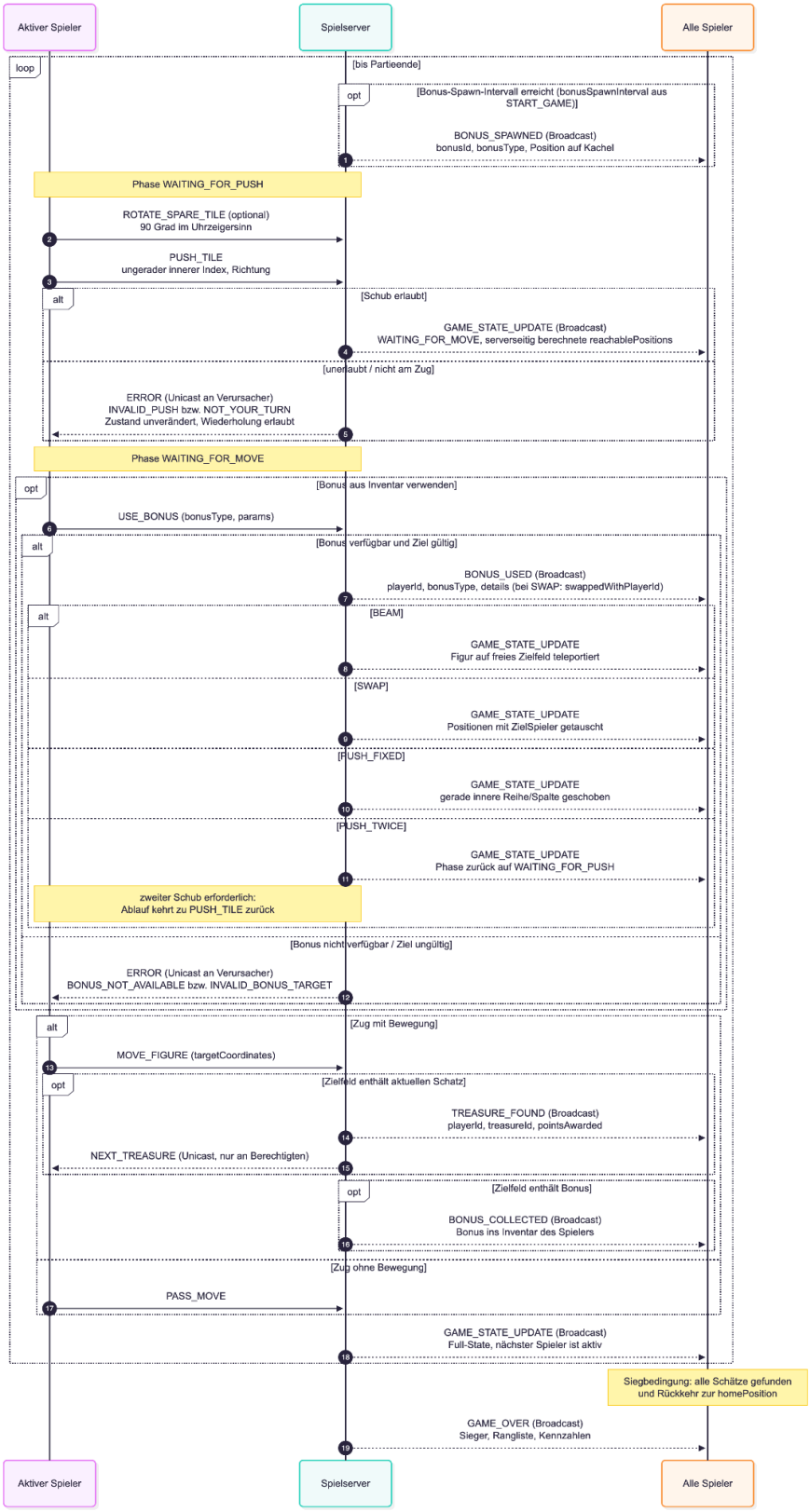


Klassendiagramm Spielserver-Modul

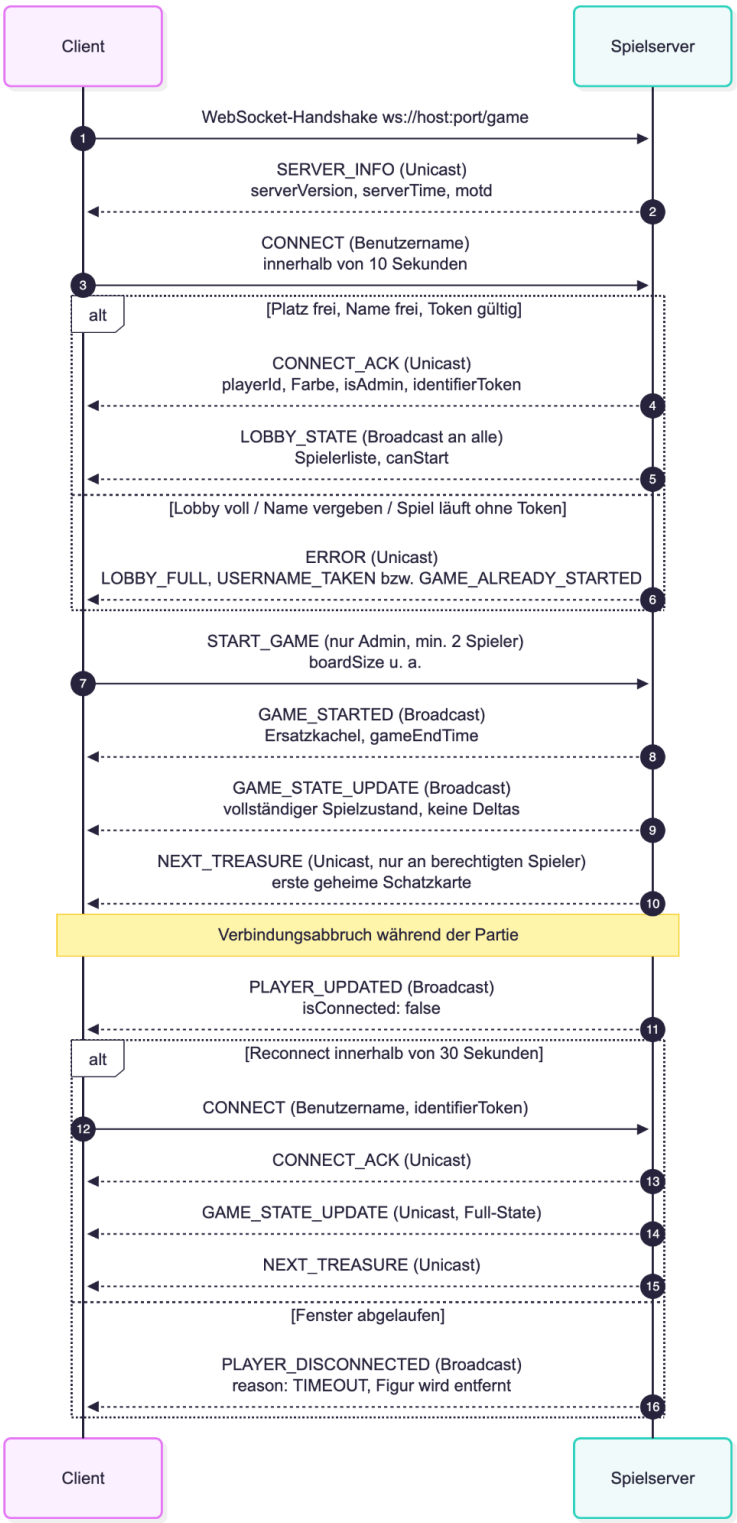




Sequenzdiagramm Interaktion zwischen Verzeichnisserver und Client und Spielserver



Sequenzdiagramm Spielzug



Sequenzdiagramm WebSocket-Verbindungsaufbau